Двухуровневая система порождения и исполнения вычислительных потоков

Архитектура, реализация и ключевые инженерные решения

Технический отчёт

Март 2026

Аннотация

В настоящей работе описана двухуровневая система `development_system`, предназначенная для интерактивного порождения и исполнения вычислительных потоков на основе больших языковых моделей. Система реализована в виде двух взаимодействующих Telegram-ботов: оркестратора (*Мастер*) на языке Racket и исполнителя (*Подмастерье*), сочетающего Racket с Common Lisp. Ключевой элемент системы — конвейер порождения: Мастер принимает от пользователя описание задачи на естественном языке, формирует структурированное задание, вызывает внешний агент Claude Code, получает синтаксически корректный Common Lisp, компилирует его средствами SBCL и сохраняет как именованный *поток*. Подмастерье загружает активный поток и исполняет его для каждого пользовательского запроса. Описаны архитектура, протоколы взаимодействия компонентов, система условий и перезапуска в Common Lisp, а также механизмы комбинирования потоков.

Содержание

1 Введение **3**
1.1 Участники системы . . . 3

2 Архитектура системы **3**
2.1 Обзор взаимодействия . . . 3
2.2 Файловая среда как протокол . . . 4

3 Конфигурация оркестратора: `dusha.rkt` **4**
3.1 Формат конфигурационного файла . . . 4
3.2 Операции над конфигурацией: `dusha.rkt` . . . 5

4 Модуль интеграции с языковой моделью: `llm.rkt` **6**

5 Конвейер порождения потоков: `potoki.rkt` **7**
5.1 Вызов Claude Code как подпроцесса . . . 7
5.2 Формирование задания . . . 7
5.3 Извлечение кода и определение имени пакета . . . 8
5.4 Верификация через компилятор SBCL . . . 9

6 Оркестратор: `мастер/main.rkt` **9**
6.1 Reply-клавиатура . . . 9
6.2 Диспетчер сообщений . . . 10
6.3 Управление состоянием «ожидания ввода» . . . 11
6.4 Авторизация . . . 11

7 Исполнитель: `подмастерье/исполнитель.lisp` **11**
7.1 Инициализация Quicklisp . . . 12
7.2 Загрузка потока . . . 12
7.3 Поиск точки входа . . . 12

8 Анатомия вычислительного потока **13**
8.1 Поток «уточнение»: итеративное решение задач . . . 13
8.2 Поток «исполнитель»: порождение и выполнение кода . . . 14

9 Комбинирование потоков: `kombinirovanie.rkt` **15**
9.1 Последовательная цепочка . . . 15

10 Протокол взаимодействия с Telegram Bot API **15**
10.1 Long polling . . . 15
10.2 Разбор обновлений . . . 16

11 Инженерные уроки и ключевые ошибки **16**
11.1 Порядок аргументов `string-prefix?` в Racket . . . 16
11.2 JSON-массивы в Racket: списки, не векторы . . . 16
11.3 Регулярные выражения: `#rx` vs `#px` . . . 17
11.4 Динамическая загрузка Quicklisp в Common Lisp . . . 17

12 Переменные окружения и конфигурация **17**

13 Метафайлы потоков: формат .мета **18**

14 Цикл опроса: обработка ошибок и перезапуск **18**
14.1 Устойчивость к сетевым сбоям 18
14.2 Защита от чужих пользователей 19

15 Запуск потока в Подмастерье: полный путь данных **19**

16 Производительность и задержки **20**
16.1 Источники задержки 20
16.2 Возможности оптимизации 21

17 Заключение **21**

Приложение: структура файлов проекта **22**

1. Введение

Взаимодействие человека с вычислительной машиной через язык естественных запросов остаётся нерешённой инженерной задачей. Существующие подходы, как правило, сводятся к одному из двух полюсов: либо к *чёрному ящику* — диалоговой системе, не способной к программной автоматизации, либо к *явному программированию* — требующему от пользователя глубокого знания API. `development_system`предлагает иной подход: **порождение кода на основе диалога** с немедленной верификацией и исполнением результата.

Центральное понятие системы — *вычислительный поток*: самодостаточная программа на Common Lisp, принимающая задачу в виде строки и возвращающая строковый ответ. Потоки порождаются по описанию на естественном языке, проверяются компилятором и выполняются в изолированном процессе SBCL. Такой подход позволяет:

- сохранять и повторно использовать задокументированную логику;
- комбинировать потоки последовательно, параллельно или условно;
- отделять роль «архитектора» (Мастер) от роли «исполнителя» (Подмастерье).

Система написана в духе SICP [1]: минимализм, рекурсия вместо итерации, маленькие чистые функции, паттерн-матчинг. Имена прикладных понятий — на русском языке.

1.1. Участники системы

Компонент	Язык	Роль
Мастер	Racket	Оркестратор: диалог, порождение потоков, управление
Подмастерье	Racket + SBCL	Исполнитель: приём задачи, запуск потока, ответ
Поток	Common Lisp	Вычислительная единица с точкой входа `выполнить`
Claude Code	Go (внешний)	Агент порождения кода, вызывается как подпроцесс

Таблица 1: Компоненты системы

2. Архитектура системы

2.1. Обзор взаимодействия

Система работает следующим образом. Мастер принимает сообщение пользователя через Telegram Bot API и классифицирует его: системная команда, кнопка меню или свободный текст. Для команды *«Породить поток»* запускается конвейер генерации. По завершении пользователь активирует созданный поток командой `/запустить`, и с этого момента все запросы к Подмастерью исполняются через данный поток.

```
Пользователь
  │
  ├── [Telegram] ──► Мастер (Racket)
  │                    │
  │                    ├── Claude Code (subprocess)
  │                    │     └── порождает .lisp
  │                    ├── SBCL (subprocess, compile-file)
  │                    │     └── верифицирует код
  │                    └── файловая система ──► активный.scm
  │
  └── [Telegram] ──► Подмастерье (Racket)
                       │
                       └── SBCL (subprocess, load + выполнить)
                             └── OpenRouter API
```

Рис. 1: Схема потоков данных

2.2. Файловая среда как протокол

Взаимодействие между Мастером и Подмастерьем осуществляется *исключительно через файловую систему*. Никакого IPC, никаких сокетов. Мастер записывает файл `потоки/активный.scm`, Подмастерье его читает при получении каждого запроса.

Формат файла — S-выражение:

```
активный
(
  имя(     "уточнение")
  файл(    "/path/totoпотокиуточнение//.lisp")
  пакет(   "потокуточнение-"))
```

Listing 1: Файл активного потока `активный.scm`

Такое решение обеспечивает полную независимость компонентов и тривиальную отладку: состояние системы всегда доступно для инспекции как обычный текстовый файл.

3. Конфигурация оркестратора: `dusha.rkt`

3.1. Формат конфигурационного файла

Поведение Мастера задаётся конфигурационным файлом `душа.scm` в формате S-выражения. Файл содержит четыре поля:

Поле	Тип	Назначение
указание	строка	System prompt для языковой модели
модель	строка	Идентификатор модели на OpenRouter
теплота	число	Temperature (0.0–2.0)
память	список	Предпочтения пользователя (user preferences)
девиз	строка	Стилевые директивы генерации кода

Таблица 2: Поля конфигурации Мастера

```
конфиг
(
  (system-prompt  "Ты помощник для создания вычислительных потоков
                   на Common Lisp.")
  модель(         "claude-sonnet-4-20250514")
  (temperature    0.3)
  (preferences    стиль(( . "рекурсия, однострочники, SICP")))
  (style-guide    "краткость, без англицизмов, функциональный стиль"))
```

Listing 2: Конфигурационный файл душа.scm

Выбор S-выражений как формата мотивирован единообразием: весь проект оперирует данными в этом формате — метафайлы потоков, файл активного потока, трасс-логи. Альтернативы (TOML, JSON, YAML) потребовали бы отдельных парсеров.

3.2. Операции над конфигурацией: dusha.rkt

Модуль dusha.rkt реализует типичный функциональный CRUD для неизменяемых S-выражений. Доступ к полю — через паттерн-матчинг:

```
(define получитьполе(- конфиг имя)
  (match конфиг
    [(list _ поля ...)          ; первый элемент — тег
   конфигдуша(//etc.)
     (for/or поле([ (in-list поля)])
       (match поле
         [(list (== имя) значение) значение]
         [_ #f]))]
    [_ #f]))
```

Listing 3: Чтение поля из S-выражения (dusha.rkt)

Обновление реализовано как структурное преобразование без мутации:

```
(define заменитьполе(- конфиг имя значение)
  (match конфиг
    [(list тег поля ...)
     (cons тег
           (for/list поле([ (in-list поля)])
             (match поле
               [(list (== имя) _) (list имя значение)]
               другое[ другое])))]))
```

Listing 4: Функциональное обновление поля конфига

Паттерн (== имя) обеспечивает структурное равенство при матчинге символа. Тег (конфиг, мета, след) хранится как первый элемент и не участвует в обходе полей — это даёт одну реализацию получить-поле для всех S-выражений проекта.

4. Модуль интеграции с языковой моделью: llm.rkt

Для диалогового режима (свободный текст пользователя) Мастер обращается к языковой модели через OpenRouter API. Модуль llm.rkt инкапсулирует этот вызов в одну функцию:

```
(define спроситьмодель(- указание модель теплота сообщения)
  (let* м((    нормализоватьмодель(- модель))
          тело( (jsexpr->bytes
                 (hash 'model м
                       'temperature теплота
                       'messages
                       (cons (hash 'role "system" 'content указание)
                             сообщения))))
          заголовки(
           (list (string-append "Authorization: Bearer "
                                ключ(-openrouter))
                 "Content-Type: application/json"))
          порт( (post-pure-port (string->url api-url) тело заголовки))
          ответ( (bytes->jsexpr (port->bytes порт))))
    (close-input-port порт)
    (let* варианты(( (hash-ref ответ 'choices #f))
            первый(   (and варианты
                           (> (vector-length варианты) 0)
                           (vector-ref варианты 0)))
            сообщ(    (and первый (hash-ref первый 'message #f)))
            текст(    (and сообщ  (hash-ref сообщ  'content #f))))
      (or текст "Нет ответа от модели."))))
```

Listing 5: Ядро интеграции с LLM (llm.rkt)

Ключевые решения в реализации:

1. **Нормализация модели**: функция нормализовать-модель приводит краткие имена вида "claude-sonnet-4" к полным "anthropic/claude-sonnet-4", ожидаемым OpenRouter.
2. **JSON в Racket**: библиотека racket/json представляет JSON-массивы как *списки*, а не векторы. Системный промпт добавляется через cons, формируя список сообщений без лишних аллокаций.
3. **Идемпотентность**: функция не хранит состояния и не выполняет побочных эффектов кроме HTTP-запроса.

5. Конвейер порождения потоков: potoki.rkt

Это центральный модуль системы, реализующий полный жизненный цикл потока: **описание → задание → код → верификация → хранение**.

5.1. Вызов Claude Code как подпроцесса

Для генерации кода используется инструмент Claude Code, запускаемый как дочерний процесс через стандартный API подпроцессов Racket:

```
(define спросить-клод(- вопрос)
  (let путь-клод((- (find-executable-path "claude")))
    (unless путь-клод-
      (error спросить-клод'- "Программа claude не найдена в PATH"))
    (let-values процесс([( выход вход ошибки)
                  (subprocess #f #f #f
                              путь-клод- "--print" вопрос)])
      (close-output-port вход)
      (let ответ(( (port->string выход)))
        (close-input-port выход)
        (close-input-port ошибки)
        (subprocess-wait процесс)
        (let код(( (subprocess-status процесс)))
          (if (zero? код) ответ
              (error спросить-клод'-
                     "Claude Code завершился с кодом ~a" код)))))))
```

Listing 6: Вызов Claude Code (potoki.rkt)

Флаг `--print` переводит Claude Code в неинтерактивный режим: агент получает запрос на stdin, выполняет его и завершается. Это делает вызов прозрачным и детерминированным с точки зрения системы.

5.2. Формирование задания

Мастер формирует задание для Claude Code, объединяя поля конфигурации (`system-prompt`, `style-guide`) с пользовательским описанием и явными техническими требованиями:

```
(define сформировать-задание(- душа описание)
  (let указание(( (or получить-поле(- душа указание') ""))
        девиз(    (or получить-поле(- душа девиз') "")))
    (string-append
     "Ты генератор вычислительных потоков на Common Lisp.\n"
     "Системное указание: " указание "\n"
     "Девиз: " девиз "\n\n"
     "Задача: " описание "\n\n"
     "Требования к коду:\n"
     "- В начале после defpackage: "
     "  (ql:quickload '(\"drakma\" \"cl-json\") :silent t)\n"
     "- HTTP: drakma:http-request; JSON: cl-json\n"
     "- Getenv: uiop:getenv (OPENROUTER_API_KEY)\n"
     "- Пакет потокимя:-<>; функция (defun выполнить задача() ...)\n"
```

```
     "- Рекурсия вместо loop; функции до 15 строк; без CLOS\n"
     "- Верни ТОЛЬКО код Common Lisp\n")))
```

Listing 7: Формирование задания для генератора кода

5.3. Извлечение кода и определение имени пакета

Ответ Claude Code может содержать markdown-обёртку (```lisp...```). Функция `извлечь-код` реализует конечный автомат над строками ответа:

```
(define извлечькод(- ответ)
  (let строки(( (string-split ответ "\n")))
    (let ищи ост(( строки) внутри( #f) собрано( '()))
      (cond
        [(null? ост)
         (if (null? собрано)
             (string-trim ответ)        ; код без обёртки
             (string-join (reverse собрано) "\n"))]
        [(and (not внутри)
              (let c(( (string-trim (car ост))))
                (or (string-prefix? c "```lisp")
                    (string-prefix? c "```common-lisp"))))
         ищи( (cdr ост) #t собрано)]   ; вошли в блок
        [(and внутри
              (string-prefix? (string-trim (car ост)) "```"))
         ищи( (cdr ост) #f собрано)]   ; вышли из блока
        внутри[
         ищи( (cdr ост) #t (cons (car ост) собрано))]
        [else
         ищи( (cdr ост) #f собрано)]))))
```

Listing 8: Извлечение кода из ответа (конечный автомат)

> **Важное замечание о `string-prefix?` в Racket.** Функция принимает аргументы в порядке `(string-prefix? строка префикс)`, то есть возвращает `#t`, если второй аргумент является префиксом первого. Это противоположно интуитивному порядку, принятому в большинстве языков, и является источником типичной ошибки при портировании кода.

Имя пакета извлекается регулярным выражением Perl-совместимого формата (`#px`):

```
(define имяпакета(- код)
  (let совп(( (regexp-match #px"поток:-([^\\s()]+)" код)))
    (if совп
        (string-trim (cadr совп))
        "безымянный")))
```

Listing 9: Извлечение имени пакета

Использование `#px` (PCRE) вместо `#rx` (базовые регулярные выражения) принципиально: только PCRE поддерживает мета-символы
`s` и
`w`, позволяющие корректно обработать пробельные символы и переносы строк.

5.4. Верификация через компилятор SBCL

Перед сохранением код компилируется в SBCL для проверки синтаксической корректности. Подпроцесс запускается с предварительной загрузкой Quicklisp и зависимостей:

```
(define проверитьпоток(- путь)
  (let* путь((-sbcl  (find-executable-path "sbcl"))
         (ql-init    (path->string
                      (build-path (find-system-path 'home-dir)
                                  "quicklisp" "setup.lisp")))
         загр(-ql    (format "(when (probe-file ~s) (load ~s :verbose
   nil))"
                             ql-init ql-init))
         загрзав(-   (string-append
                      "(when (find-package :ql)"
                      "  (funcall (intern \"QUICKLOAD\" :ql)"
                      "   '(\"drakma\" \"cl-json\" \"uiop\")"
                      "   :silent t))"))
         выражение(  (format
                      "(multiple-value-bind (fasl warn fail)"
                      "    (compile-file ~s)"
                      "  (if fail (sb-ext:exit :code 1)"
                      "           (sb-ext:exit :code 0)))"
                      (path->string путь))))
    (let-values проц([( выход вход ошибки)
                  (subprocess #f #f #f путь-sbcl
                              "--noinform" "--non-interactive"
                              "--eval" загр-ql
                              "--eval" загрзав-
                              "--eval" выражение)])
      (close-output-port вход)
      (let вывод(( (port->string выход))
            ош(    (port->string ошибки)))
        (close-input-port выход)
        (close-input-port ошибки)
        (subprocess-wait проц)
        (values (zero? (subprocess-status проц))
                (string-append вывод ош))))))
```

Listing 10: Верификация кода в SBCL (potoki.rkt)

Функция `compile-file` возвращает три значения: путь к `.fasl`-файлу, флаг предупреждений и флаг ошибок. Именно третье значение (`fail`) определяет результат верификации. Код завершения подпроцесса транслируется обратно в булевое значение.

6. Оркестратор: `мастер/main.rkt`

6.1. Reply-клавиатура

Пользовательский интерфейс реализован через Telegram Reply Keyboard — постоянную клавиатуру, отображаемую в нижней части экрана мессендже-

ра. Клавиша генерирует текстовое сообщение, которое обрабатывается в обычном диспетчере:

```
(define кнопка(-reply текст)
  (hash 'text текст))

(define главноеменю(--reply)
  (list
   (list кнопка(-reply "□ Породить поток")
         кнопка(-reply "□ Мои потоки"))
   (list кнопка(-reply "□□ Состояние")
         кнопка(-reply "□ Остановить"))
   (list кнопка(-reply "□ Душа")
         кнопка(-reply "□ Память"))
   (list кнопка(-reply "□ Время"))))

(define послатьскнопками(-- идчата- текст)
  послатьзапрос(- "sendMessage"
    (jsexpr->string
      (hash 'chat_id      идчата-
            'text         текст
            'reply_markup
            (hash 'keyboard        главноеменю(--reply)
                  'resize_keyboard #t
                  'persistent      #t)))))
```

Listing 11: Reply-клавиатура (main.rkt)

Флаг persistent обеспечивает постоянное присутствие клавиатуры даже после перезапуска клиента.

6.2. Диспетчер сообщений

Диспетчер сопоставляет входящий текст с командами, включая русскоязычные кнопки меню:

```
(define сформироватьответ(- текст)
  (cond
    [(or (string=? текст "/start")
         (string=? текст "меню/"))         #f]  ; показать меню
    [(or (string=? текст "□ Время")
         (string=? текст "время/"))        форматвремя(-)]
    [(or (string=? текст "□□ Состояние")
         (string=? текст "состояние/"))    обработатьсостояние(-)]
    [(or (string=? текст "□ Мои потоки")
         (string=? текст "потоки/"))       обработатьпотоки(-)]
    [(or (string=? текст "□ Остановить")
         (string=? текст "остановить/"))   обработатьостановить(-)]
    [(string-prefix? текст "породить/ ")
     обработатьпородить(- (substring текст 10))]
    [else обработатьдиалог(- текст)]))
```

Listing 12: Диспетчер команд

6.3. Управление состоянием «ожидания ввода»

Кнопка «Породить поток» требует последующего ввода описания. Для этого используется мутируемая хеш-таблица, отслеживающая чаты, находящиеся в режиме ввода:

```
(define ожидаетпородить*-* (make-hash))

;; В цикле опроса:
(cond
  [(string=? текст "□ Породить поток")
   (hash-set! ожидаетпородить*-* идчата- #t)
   послатьсообщение(- идчата-
     "Опишите задачу для нового потока:")]
  [(hash-ref ожидаетпородить*-* идчата- #f)
   (hash-remove! ожидаетпородить*-* идчата-)
   (let p(( обработатьпородить(- текст)))
     послатьскнопками(-- идчата- p))]
  [else ...])
```

Listing 13: Двухшаговый ввод через состояние

6.4. Авторизация

Оба бота ограничены одним администратором. Идентификатор задаётся через переменную окружения `ADMIN_CHAT_ID`:

```
(define идадмина(-)
  (let ((s (getenv "ADMIN_CHAT_ID")))
    (and s (string->number s))))

(define админ(? идчата-)
  (let a(( идадмина(-)))
    (or (not a) (= идчата- a))))

;; В цикле опроса: игнорируем чужих молча
(when (and админ(? идчата-) (eq? тип 'message))
  ... ; обработка)
```

Listing 14: Проверка авторизации

Если переменная не задана, авторизация отключена (`(not a) = #t`). Это позволяет запустить систему без конфигурации в режиме разработки.

7. Исполнитель: `подмастерье/исполнитель.lisp`

Common Lisp исполнитель реализует загрузку потока, его верификацию, выполнение задачи и обработку условий. Это единственный компонент системы на Common Lisp (не считая самих потоков).

7.1. Инициализация Quicklisp

Поскольку SBCL запускается как чистый процесс без пользовательской инициализации, необходима явная загрузка Quicklisp перед выполнением потока:

```
(defun обеспечить-quicklisp ()
  "Загружает Quicklisp если ещё не загружен."
  (unless (find-package :ql)
    (let ((ql-init (merge-pathnames "quicklisp/setup.lisp"
                                    (user-homedir-pathname))))
      (when (probe-file ql-init)
        (load ql-init :verbose nil :print nil))))
  (when (find-package :ql)
    (funcall (intern "QUICKLOAD" :ql)
             '("drakma" "cl-json" "flexi-streams" "uiop")
             :silent t)))
```

Listing 15: Инициализация Quicklisp в исполнителе

Использование `intern` вместо прямого символа `ql:quickload` обязательно: пакет `:ql` ещё не существует в момент компиляции файла.

7.2. Загрузка потока

```
(defun загрузитьпоток- путь()
  "Загружает файл потока в текущий образ."
  (restart-case
      (progn
        (unless (probe-file путь)
          (error "Файл потока не найден: ~a" путь))
        обеспечить(-quicklisp)
        (load путь :verbose nil :print nil)
        t)
    пропустить( ()
      :report "Пропустить загрузку потока"
      nil)))
```

Listing 16: Загрузка потока с обработкой условий

7.3. Поиск точки входа

Потоки могут иметь произвольные имена пакетов, поэтому точка входа ищется по соглашению об именовании — функция `ВЫПОЛНИТЬ` в заданном пакете:

```
(defun найтиточкувхода-- имяпакета(-)
  "Ищет функцию ВЫПОЛНИТЬ в указанном пакете."
  (let пакет(( (find-package (string-upcase имяпакета-))))
    (when пакет
      (let символ(( (find-symbol "ВЫПОЛНИТЬ" пакет)))
        (when (and символ (fboundp символ))
          (symbol-function символ))))))
```

Listing 17: Динамический поиск точки входа

`string-upcase` необходим: в Common Lisp символы хранятся в верхнем регистре, а имена пакетов в метафайлах написаны строчными буквами.

8. Анатомия вычислительного потока

Потоки генерируются по единому шаблону и имеют предсказуемую структуру. Рассмотрим два реальных потока, порождённых системой.

8.1. Поток «уточнение»: итеративное решение задач

Поток реализует паттерн *iterative refinement*: каждый шаг улучшает ответ предыдущего, останавливаясь при достижении критерия качества или по исчерпании лимита итераций.

```
(defun уточнить задача( предыдущий шаг)
  "Рекурсивно уточняет ответ. Шаг от 1 до 5."
  (if (> шаг 5)
      предыдущий
      (let* подсказка((
               (if (string= предыдущий "")
                   (format nil "Задача: ~a" задача)
                   (format nil
                     "Задача: ~aПредыдущий~% ответ:~%~aУточни~%~%."
                     задача предыдущий)))
             история( (list наставление()
                            сообщение( "user" подсказка)))
             ответ( послатьзапрос(- история)))
        (if достаточноли(- ответ)
            ответ
            уточнить( задача ответ (1+ шаг))))))
```

Listing 18: Поток уточнения: ядро алгоритма

Критерий остановки:

```
(defun достаточноли- ответ()
  "Проверяет признаки завершённости в тексте."
  (and (> (length ответ) 20)
       (or (search "ГОТОВО" ответ)
           (search "Итог:" ответ)
           (search "Ответ:" ответ))))
```

Listing 19: Критерий качества ответа

Точка входа оборачивает рекурсию в `restart-case`:

```
(defun выполнить задача()
  "Итеративно решает задачу за -15 шагов."
  (restart-case
      уточнить( задача "" 1)
    вернутьошибку(- (e)
      :report "Вернуть описание ошибки"
      (format nil "Ошибка: ~a" e))))
```

Listing 20: Точка входа потока уточнения

8.2. Поток «исполнитель»: порождение и выполнение кода

Поток реализует полный цикл: получить задачу → попросить модель написать Python-код → очистить его от markdown-обёртки → запустить через `uiop:run-program`.

```
(defun исполнить-python код()
  (restart-case
      (multiple-value-bind вывод( кодошибки- статус)
          (uiop:run-program
           (list "python3" "-c" код)
           :output        :string
           :error-output  :string
           :ignore-error-status t)
        (declare (ignore статус))
        (if (and кодошибки- (string/= кодошибки- ""))
            (values nil кодошибки-)
            (values вывод nil)))
    вернутьошибку(- (e)
      (values nil (format nil "Ошибка: ~A" e)))))
```

Listing 21: Исполнение Python-кода через uiop

Очистка кода от markdown удаляет строки, начинающиеся с ```:

```
(defun убратьобёртку- строки()
  (cond ((null строки) nil)
        начинаетсяс((- "```" (car строки))
         убратьобёртку(- (cdr строки)))
        (t (cons (car строки)
                 убратьобёртку(- (cdr строки))))))

(defun начинаетсяс- префикс( строка)
  (and (>= (length строка) (length префикс))
       (string= префикс строка :end2 (length префикс))))
```

Listing 22: Рекурсивная очистка markdown-обёртки

Точка входа объединяет все шаги:

```
(defun выполнить задача()
  (restart-case
      (let* код((-python запроситьмодель(- задача))
                чистыйкод(- очиститькод(- код-python)))
        (multiple-value-bind результат( ошибка)
            исполнить(-python чистыйкод-)
          (or ошибка результат)))
    отказ( причина()
      (format nil "Отказ: ~A" причина))))
```

Listing 23: Точка входа потока исполнителя

9. Комбинирование потоков: kombinirovanie.rkt

Мастер поддерживает три базовые схемы комбинирования потоков и произвольную четвёртую, генерируемую по описанию.

9.1. Последовательная цепочка

Порождает новый поток-обёртку, последовательно передающий результат:

```
(define шаблонцепочки(- имена пакеты файлы)
  (string-append
   "(defpackage поток:-" (string-join имена "-") " (:use :cl))\n"
   "(in-package поток:-" (string-join имена "-") ")\n\n"
   ;; загрузка всех потоков
   (apply string-append
          (map (lambda ф()
                 (format "(load ~s :verbose nil :print nil)\n" ф))
               файлы))
   "\n(defun выполнить задача()\n"
   "  (let цикл вх(( задача)\n"
   "             ост( (list " (string-join
                                (map (lambda п()
                                       (string-append "'" п
   "выполнить:"))
                                     пакеты)
                                " ") )))\n"
   "    (if (null ост) вх\n"
   "        цикл( (funcall (car ост) вх) (cdr ост)))))\n"))
```

Listing 24: Генерация кода цепочки потоков

10. Протокол взаимодействия с Telegram Bot API

10.1. Long polling

Оба бота используют long polling: запрос `getUpdates` с `timeout=30` блокируется на сервере Telegram до появления нового обновления или истечения тайм-аута.

```
(define получитьобновления(- сдвиг)
  (let* адрес(( (string->url
                 (string-append адрес(-api "getUpdates")
                                "?timeout=30&offset="
                                (number->string сдвиг))))
         порт( (get-pure-port адрес)))
    (begin0 (port->string порт)
            (close-input-port порт))))
```

Listing 25: Long polling в Racket

Параметр `offset` содержит `update_id + 1` последнего обработанного обновления, что исключает повторную обработку.

10.2. Разбор обновлений

Парсер обрабатывает два типа событий: обычные сообщения и ответы на callback-кнопки. Результат унифицирован в единый формат события:

```
(define извлечьсобытия(- обновления)
  (filter-map
   (lambda обн()
     (cond
       [(hash-has-key? обн 'message)
        (let* сообщ((   (hash-ref обн 'message))
               идобн(-  (hash-ref обн 'update_id 0))
               чат(     (hash-ref сообщ 'chat (hash)))
               идчата(- (hash-ref чат 'id 0))
               текст(   (hash-ref сообщ 'text "")))
          (list идобн- идчата- 'message текст))]
       [(hash-has-key? обн 'callback_query)
        (let* ((cb      (hash-ref обн 'callback_query))
               идобн(-  (hash-ref обн 'update_id 0))
               чат(     (hash-ref
                          (hash-ref cb 'message (hash))
                          'chat (hash)))
               идчата(- (hash-ref чат 'id 0))
               (cb-id   (hash-ref cb 'id ""))
               данные(  (hash-ref cb 'data "")))
          (list идобн- идчата- 'callback cb-id данные))]
       [else #f]))
   обновления))
```

Listing 26: Парсер обновлений: унификация событий

11. Инженерные уроки и ключевые ошибки

В процессе разработки были выявлены несколько нетривиальных особенностей применяемых инструментов:

11.1. Порядок аргументов `string-prefix?` в Racket

> Функция `(string-prefix? s1 s2)` в `racket/string` возвращает `#t` если *s2* является префиксом *s1*. Иными словами, первый аргумент — строка, второй — искомый префикс. Это противоположно семантике аналогичных функций в Python, Haskell и Go.

Проверка: `(string-prefix? "/породить тест" "/породить ")` → `#t`.

11.2. JSON-массивы в Racket: списки, не векторы

Библиотека `racket/json` представляет JSON-массивы как списки Racket, а не векторы. Передача вектора в `jsexpr->bytes` вызывает ошибку типа. Корректное формирование массива сообщений:

```
;; Неправильно (vector):
'messages (list->vector ...)   ; → ошибка jsexpr->bytes

;; Правильно (list):
'messages (cons (hash 'role "system" 'content указание)
                сообщения)
```

Listing 27: Правильное формирование JSON-массива

11.3. Регулярные выражения: #rx vs #px

В Racket существуют два типа регулярных выражений: #rx (POSIX-совместимые) не обрабатывают
s,
w и аналогичные мета-символы. Для работы с ними необходим #px (PCRE):

```
;; #rx не понимает \s:
(regexp-match #rx"поток:-([^\s()]+)" код)   ; ошибка

;; #px работает корректно:
(regexp-match #px"поток:-([^\\s()]+)" код)  ; OK
```

11.4. Динамическая загрузка Quicklisp в Common Lisp

При вызове ql:quickload в файле, загружаемом до инициализации Quicklisp, компилятор сигнализирует об ошибке «Package QL does not exist». Обходное решение — intern:

```
;; Неправильно ошибка( компиляции до загрузки Quicklisp):
(ql:quickload '("drakma") :silent t)

;; Правильно откладывает( разрешение символа на время выполнения):
(funcall (intern "QUICKLOAD" :ql) '("drakma") :silent t)
```

Listing 28: Безопасная загрузка через intern

12. Переменные окружения и конфигурация

Переменная	Компонент	Назначение
MASTER_BOT_TOKEN	Мастер	Токен Telegram-бота Мастера
APPRENTICE_BOT_TOKEN	Подмастерье	Токен Telegram-бота Подмастерья
OPENROUTER_API_KEY	Оба	Ключ API OpenRouter
ADMIN_CHAT_ID	Оба	Telegram ID единственного пользователя

Таблица 3: Переменные окружения системы

13. Метафайлы потоков: формат .мета

Каждый поток сопровождается файлом метаданных в формате S-выражения. Это решение продолжает сквозную тему проекта: состояние системы — всегда читаемый текст.

```
мета
(
  имя(          "уточнение")
  создан(       "2026-03-13T10:17:12")
  описание(     "Итеративное решение задач через языковую модель")
  зависимости( ())
  входитв(-     ())   ; список потоковагрегаторов-
  состояние(    готов:))  ; готов: | работает: | ошибка:
```

Listing 29: Формат файла метаданных потока

Доступ к полям метафайла через паттерн-матчинг аналогичен получить-поле из dusha.rkt:

```
(define полеметы(- мета имя)
  (match мета
    [(list мета' поля ...)
     (for/or поле([ (in-list поля)])
       (match поле
         [(list (== имя) значение) значение]
         [_ #f]))]
    [_ #f]))

;; Пример использования:
(define описаниепотока(- имя)
  (let* путьмета((- (build-path каталог
                                (string-append имя "мета.")))
         мета(      прочитатьмету(- путьмета-)))
    (or (and мета полеметы(- мета описание'))
        "без описания")))
```

Listing 30: Чтение метаданных потока

14. Цикл опроса: обработка ошибок и перезапуск

14.1. Устойчивость к сетевым сбоям

Главный цикл опроса в обоих ботах оборачивает каждую итерацию в with-handlers. При любом исключении — сетевом тайм-ауте, ошибке парсинга JSON, недоступности API — цикл засыпает на пять секунд и продолжает работу с тем же сдвигом:

```
(define запуститьопрос(-)
  загрузитьконфиг(-)      ; читает душа.scm → конфиг**
  (displayln "Мастер запущен. Ожидаю сообщения...")
  (let цикл сдвиг(( 0))
```

```
    (with-handlers
        ([exn:fail?
          (lambda (e)
            (displayln
              (format "Ошибка: ~a" (exn-message e)))
            (sleep 5)
            цикл( сдвиг))])   ; повторяем с тем же сдвигом
      (let* ответ((    получитьобновления(- сдвиг))
             события(  извлечьсобытия(-
                          разобратьобновления(- ответ))))
        (let обр ост(( события) макс( сдвиг))
          (if (null? ост)
              цикл( макс)
              (let* событие(( (car ост))
                     идобн(-  (first событие))
                     ...)
                ...
                обр( (cdr ост)
                     (max макс (+ идобн- 1)))))))))
```

Listing 31: Устойчивый цикл опроса с перезапуском

Ключевой инвариант: сдвиг не обновляется до успешной обработки всех сообщений текущего батча. Это гарантирует, что при падении в середине батча все сообщения будут обработаны повторно.

14.2. Защита от чужих пользователей

Проверка авторизации происходит до любой обработки события, чтобы не расходовать ресурсы LLM на нежелательные запросы:

```
(when (and админ(? идчата-) (eq? тип 'message))
  ;; только здесь происходит реальная обработка
  (let текст(( (fourth событие)))
    ...))
;; чужие молча игнорируются — никакого ответа
```

Listing 32: Раннее отклонение неавторизованных

15. Запуск потока в Подмастерье: полный путь данных

Цепочка вызовов при исполнении пользовательского запроса в Подмастерье охватывает несколько уровней абстракции: от Telegram до SBCL-подпроцесса.

```
(define выполнитьв(--sbcl путьпотока- имяпакета- задача)
  (let* путь((-sbcl (find-executable-path "sbcl"))
         исп(       (->string путьисполнителя(-)))
         загрузка(
          (format "(load ~s :verbose nil :print nil)" исп))
         вызов(
          (format
           "(let ф(( (funcall (intern ЗАПУСК\"\" ИСПОЛНИТЕЛЬ\"\")
                              ~s ~s ~s)))
```

```
              (princ ϕ) (finish-output)
              (sb-ext:exit :code 0))"
           (->string путьпотока-) имяпакета- задача)))
    (let-values проц([( выход вход ошибки)
                   (subprocess #f #f #f
                               путь-sbcl
                               "--noinform" "--non-interactive"
                               "--eval" загрузка
                               "--eval" вызов)])
      (close-output-port вход)
      (let вывод(( (port->string выход))
            ош(    (port->string ошибки)))
        (close-input-port выход)
        (close-input-port ошибки)
        (subprocess-wait проц)
        (if (zero? (subprocess-status проц))
            (string-trim вывод)
            (format "Ошибка SBCL: ~a" (string-trim ош)))))))
```

Listing 33: Исполнение потока: Racket-часть Подмастерья

Функция ЗАПУСК в пакете ИСПОЛНИТЕЛЬ загружает поток, находит точку входа и вызывает её:

```
(defun запуск путьпотока(- имяпакета- задача)
  "Загружает поток, находит точку входа, исполняет."
  загрузитьпоток(- путьпотока-)
  (let точкавхода((- найтиточкувхода(-- имяпакета-)))
    (unless точкавхода-
      (error "Функция ВЫПОЛНИТЬ не найдена в пакете ~a"
             имяпакета-))
    выполнитьсповтором(-- точкавхода- задача)))

(defun выполнитьсповтором-- функция( задача)
  "Вызывает функцию потока с защитой от условий."
  (restart-case
      (funcall функция задача)
    вернутьзадачу(- ()
      :report "Вернуть исходную задачу при ошибке"
      задача)))
```

Listing 34: Функция ЗАПУСК в исполнителе (CL)

16. Производительность и задержки

16.1. Источники задержки

Время ответа системы складывается из нескольких компонентов:

Этап	Типичное время	Примечание
Получение обновления (long poll)	<100 мс	при наличии сообщения
Запуск SBCL подпроцесса	1–3 с	холодный старт
Загрузка Quicklisp + зависимостей	2–5 с	drakma, cl-json
Обращение к OpenRouter API	1–5 с	зависит от модели
Итого (исполнение потока)	4–13 с	
Порождение потока (Claude Code)	15–60 с	однократно

Таблица 4: Типичные задержки компонентов

Наибольший вклад вносит запуск SBCL: каждый запрос к Подмастерью создаёт новый процесс. Потенциальное улучшение — постоянный SBCL-образ с предзагруженными зависимостями (`save-lisp-and-die`).

16.2. Возможности оптимизации

1. **Образы SBCL**: сохранение образа с загруженными потоком и зависимостями сокращает холодный старт с 4–8 с до <100 мс.
2. **Постоянный процесс-исполнитель**: вместо запуска нового SBCL для каждого запроса можно поддерживать долгоживущий процесс с SWANK-сервером для подачи задач.
3. **Кеширование зависимостей**: предкомпиляция `.fasl` файлов устраняет повторную компиляцию `drakma` при каждом запросе.

17. Заключение

В работе описана система `development_system`, реализующая концепцию *порождаемого кода как конфигурации*. Основные результаты:

1. **Конвейер генерации**: описание на естественном языке преобразуется в верифицированный Common Lisp код за одно обращение к языковой модели.
2. **Двухуровневая архитектура**: разделение ролей оркестратора (Racket, диалог и управление) и исполнителя (SBCL, вычисления) позволяет независимо масштабировать каждую компоненту.
3. **Файловая среда как протокол**: отсутствие явного IPC между компонентами упрощает отладку и обеспечивает персистентность состояния.
4. **Система условий Common Lisp**: паттерн `restart-case` обеспечивает структурированную обработку ошибок без исключений.
5. **Стиль SICP**: рекурсия, маленькие функции, паттерн-матчинг и идиоматический Racket обеспечивают высокую читаемость при минимальном объёме кода.

Открытые вопросы: персистентность истории диалога между сессиями, параллельное исполнение потоков с разделением результатов, формальная верификация потоков средствами ACL2.

Приложение: структура файлов проекта

```
development_system/├──
 мастер/│
   ├── main.rkt           # Оркестратор: цикл опроса, меню, диспетчер│
   ├── dusha.rkt          # Конфигурация: чтениезапись/ Sвыражений-│
   ├── potoki.rkt         # Конвейер порождения и проверки потоков│
   ├── kombinirovanie.rkt # Комбинирование: цепочка, параллель, ветвление│
   ├── llm.rkt            # Интеграция с OpenRouter API│
   ├── душа.scm           # Конфигурационный файл системы│
   ├── история/           # Журналы диалогов по( дням, .txt)│
   └── потоки/│
       ├── активный.scm   # Активный поток для Подмастерья│
       ├── уточнение.lisp # Поток: итеративное уточнение ответа│
       ├── уточнениемета. # Метаданные потока│
       ├── исполнитель.lisp # Поток: генерация и выполнение кода│
       └── исполнительмета.│├──

 подмастерье/│
   ├── main.rkt           # Ботисполнитель-: цикл опроса, диспетчер│
   └── исполнитель.lisp   # CLядро-: загрузка, запуск, condition/restart│└──

 docs/
    ├── plans/
    │   └── TODO.md        # Исходное техническое задание
    └── paper.tex          # Настоящий документ
```

Listing 35: Полная структура каталогов development_system

Общий объём кода системы (без генерируемых потоков):

Файл	**Строк**	**Язык**
мастер/main.rkt	∼330	Racket
мастер/potoki.rkt	∼270	Racket
мастер/dusha.rkt	∼120	Racket
мастер/kombinirovanie.rkt	∼160	Racket
мастер/llm.rkt	∼40	Racket
подмастерье/main.rkt	∼260	Racket
подмастерье/исполнитель.lisp	∼100	Common Lisp
Итого	∼1280	

Таблица 5: Объём кода системы

Список литературы

[1] Abelson, H., Sussman, G.J. *Structure and Interpretation of Computer Programs*. MIT Press, 1996.

[2] SBCL Team. *SBCL User Manual*. https://www.sbcl.org/manual/

[3] Flatt, M., PLT. *Reference: Racket*. https://docs.racket-lang.org/reference/

[4] OpenRouter. *OpenRouter API Documentation*. https://openrouter.ai/docs

[5] Telegram. *Bot API Reference*. https://core.telegram.org/bots/api